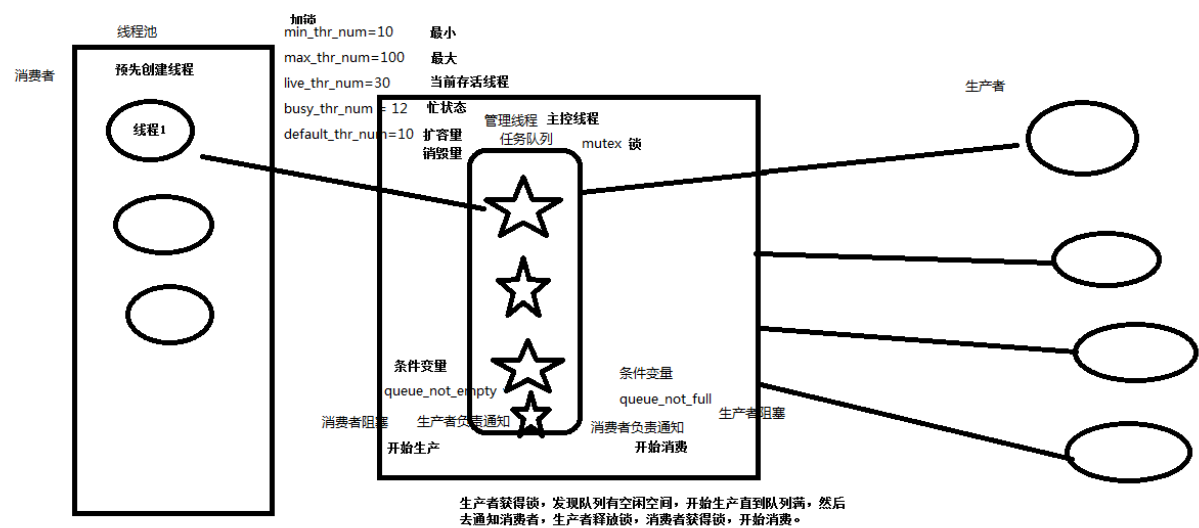


线程池

线程池并发服务器

- 1、预先创建阻塞于accept多线程，使用互斥锁上锁保护accept;
- 2、预先创建多线程，由主线程调用accept.

1、线程池的生产者和消费者关系



2、线程池代码

大多数的网络服务器，包括Web服务器都具有一个特点，就是单位时间内必须处理数目巨大的连接请求，但是处理时间却是比较短的。在传统的多线程服务器模型中是这样实现的：一旦有个请求到达，就创建一个新的线程，由该线程执行任务，任务执行完毕之后，线程就退出。这就是"即时创建，即时销毁"的策略。尽管与创建进程相比，创建线程的时间已经大大的缩短，但是如果提交给线程的任务是执行时间较短，而且执行次数非常频繁，那么服务器就将处于一个不停的创建线程和销毁线程的状态。这笔开销是不可忽略的，尤其是线程执行的时间非常非常短的情况。

线程池就是为了解决上述问题的，它的实现原理是这样的：在应用程序启动之后，就马上创建一定数量的线程，放入空闲的队列中。这些线程都是处于阻塞状态，这些线程只占一点内存，不占用CPU。当任务到来后，线程池将选择一个空闲的线程，将任务传入此线程中运行。当所有的线程都处在处理任务的时候，线程池将自动创建一定的数量的新线程，用于处理更多的任务。执行任务完成之后线程并不退出，而是继续在线程池中等待下一次任务。当大部分线程处于阻塞状态时，线程池将自动销毁一部分的线程，回收系统资源。

2.1、threadpool.h

```
1 #ifndef __THREADPOOL_H_
2 #define __THREADPOOL_H_
3
4
5 typedef struct threadpool_t threadpool_t;
6
7
8 /**
9  * @function threadpool_create
10  * @desc Creates a threadpool_t object.
11  * @param thr_num thread num
12  * @param max_thr_num max thread size
13  * @param queue_max_size size of the queue.
14  * @return a newly created thread pool or NULL
15  */
16 threadpool_t *threadpool_create(int min_thr_num, int max_thr_num, int queue_max_size)
17
18
19 /**
20  * @function threadpool_add
21  * @desc add a new task in the queue of a thread pool
22  * @param pool Thread pool to which add the task.
23  * @param function Pointer to the function that will perform the task.
24  * @param argument Argument to be passed to the function.
25  * @return 0 if all goes well, else -1
26  */
27 int threadpool_add(threadpool_t *pool, void*(*function)(void *arg), void *arg);
28
29
30 /**
31  * @function threadpool_destroy
32  * @desc Stops and destroys a thread pool.
33  * @param pool Thread pool to destroy.
34  * @return 0 if destroy success else -1
35  */
36 int threadpool_destroy(threadpool_t *pool);
37
38
39 /**
40  * @desc get the thread num
```

```

41  * @pool pool threadpool
42  * @return # of the thread
43  */
44  int threadpool_all_threadnum(threadpool_t *pool);
45
46
47  /**
48   * desc get the busy thread num
49   * @param pool threadpool
50   * return # of the busy thread
51   */
52  int threadpool_busy_threadnum(threadpool_t *pool);
53
54
55  #endif

```

2.2、threadpool.c

```

1  /*by wenpeng.xing@gmail.com*/
2
3
4  #include <stdlib.h>
5  #include <pthread.h>
6  #include <unistd.h>
7  #include <assert.h>
8  #include <stdio.h>
9  #include <string.h>
10 #include <signal.h>
11 #include <errno.h>
12 #include "threadpool.h"
13
14
15 #define DEFAULT_TIME 10          /*10s检测一次*/
16 #define MIN_WAIT_TASK_NUM 10    /*如果queue_size > MIN_WAIT_TASK_NUM 添加新的*/
17 #define DEFAULT_THREAD_VARY 10 /*每次创建和销毁线程的个数*/
18 #define true 1
19 #define false 0
20
21

```

```

22 typedef struct
23 {
24     void (*function)(void *);          /*函数指针，回调函数*/
25     void *arg;                        /*上面函数的参数*/
26 } threadpool_task_t;                 /*任务结构体*/
27
28
29 struct threadpool_t
30 {
31     pthread_mutex_t lock;              /*用于锁住当前这个结构体taskpoll*/
32     pthread_mutex_t thread_counter;    /*记录忙状态线程个数*/
33     pthread_cond_t queue_not_full;     /*当任务队列满时，添加任务的线程阻塞，等待此条件*/
34     pthread_cond_t queue_not_empty;    /*任务队列里不为空时，通知等待任务的线程*/
35     pthread_t *threads;                /*保存工作线程tid的数组*/
36     pthread_t adjust_tid;              /*管理线程tid*/
37     threadpool_task_t *task_queue;     /*任务队列*/
38     int min_thr_num;                   /*线程组内默认最小线程数*/
39     int max_thr_num;                   /*线程组内默认最大线程数*/
40     int live_thr_num;                  /*当前存活线程个数*/
41     int busy_thr_num;                  /*忙状态线程个数*/
42     int wait_exit_thr_num;             /*要销毁的线程个数*/
43     int queue_front;                   /*队头索引下标*/
44     int queue_rear;                    /*队末索引下标*/
45     int queue_size;                    /*队中元素个数*/
46     int queue_max_size;                /*队列中最大容纳个数*/
47     int shutdown;                      /*线程池使用状态，true或false*/
48 };
49
50
51 /**
52  * @function void *threadpool_thread(void *threadpool)
53  * @desc the worker thread
54  * @param threadpool the pool which own the thread
55  */
56 void *threadpool_thread(void *threadpool);
57 /**
58  * @function void *adjust_thread(void *threadpool);
59  * @desc manager thread
60  * @param threadpool the threadpool
61  */
62 void *adjust_thread(void *threadpool);
63 /**
64  * check a thread is alive
65  */

```

```

66 int is_thread_alive(pthread_t tid);
67
68
69 int threadpool_free(threadpool_t *pool);
70
71
72 threadpool_t *threadpool_create(int min_thr_num, int max_thr_num, int queue_max_size
73 {
74     int i;
75     threadpool_t *pool = NULL;
76     do{
77         if((pool = (threadpool_t *)malloc(sizeof(threadpool_t))) == NULL)
78         {
79             printf("malloc threadpool fail");
80             break;          /*跳出do while*/
81         }
82
83
84         pool->min_thr_num = min_thr_num;
85         pool->max_thr_num = max_thr_num;
86         pool->busy_thr_num = 0;
87         pool->live_thr_num = min_thr_num;
88         pool->queue_size = 0;
89         pool->queue_max_size = queue_max_size;
90         pool->queue_front = 0;
91         pool->queue_rear = 0;
92         pool->shutdown = false;
93
94
95         pool->threads = (pthread_t *)malloc(sizeof(pthread_t)*max_thr_num);
96         if (pool->threads == NULL)
97         {
98             printf("malloc threads fail");
99             break;
100         }
101         memset(pool->threads, 0, sizeof(pool->threads)*max_thr_num);
102
103
104         pool->task_queue = (threadpool_task_t *)malloc(sizeof(threadpool_task_t)*que
105         if (pool->task_queue == NULL)
106         {
107             printf("malloc task_queue fail");
108             break;

```

```

109     }
110     //对锁和条件变量初始化
111     if (pthread_mutex_init(&(pool->lock), NULL) != 0
112         || pthread_mutex_init(&(pool->thread_counter), NULL) != 0
113         || pthread_cond_init(&(pool->queue_not_empty), NULL) != 0
114         || pthread_cond_init(&(pool->queue_not_full), NULL) != 0)
115     {
116         printf("init the lock or cond fail");
117         break;
118     }
119
120
121     /* 启动min_thr_num个work thread */
122     for (i = 0; i < min_thr_num; i++)
123     {
124         pthread_create(&(pool->threads[i]), NULL, threadpool_thread, (void *)pool);
125         printf("start thread 0x%x...\n", (unsigned int)pool->threads[i]);
126     }
127     //主控线程
128     pthread_create(&(pool->adjust_tid), NULL, adjust_thread, (void *)pool);
129     return pool;
130
131
132 }while(0);
133
134
135 threadpool_free(pool);      /*前面代码调用失败时，释放pool存储空间*/
136 return NULL;
137 }
138
139
140 int threadpool_add(threadpool_t *pool, void>(*function)(void *arg), void *arg)
141 {
142     pthread_mutex_lock(&(pool->lock));
143
144
145     //任务满了
146     while ((pool->queue_size == pool->queue_max_size) && (!pool->shutdown))
147     {
148         //等待不满的条件变量，唤醒
149         pthread_cond_wait(&(pool->queue_not_full), &(pool->lock));
150     }
151     if (pool->shutdown)
152     {

```

```

153     pthread_mutex_unlock(&(pool->lock));
154 }
155
156
157 /*添加任务到任务队列里*/
158 if (pool->task_queue[pool->queue_rear].arg != NULL)
159 {
160     free(pool->task_queue[pool->queue_rear].arg);
161     pool->task_queue[pool->queue_rear].arg = NULL;
162 }
163 //添加任务
164 pool->task_queue[pool->queue_rear].function = function;
165 pool->task_queue[pool->queue_rear].arg = arg;
166 pool->queue_rear = (pool->queue_rear + 1)%pool->queue_max_size;
167 pool->queue_size++;
168
169
170 /*任务队列不为空，唤醒等待处理任务的线程*/
171 pthread_cond_signal(&(pool->queue_not_empty));
172 pthread_mutex_unlock(&(pool->lock));
173
174
175 return 0;
176 }
177
178
179 void *threadpool_thread(void *threadpool)
180 {
181     threadpool_t *pool = (threadpool_t *)threadpool; //数据
182     threadpool_task_t task;
183     while(true)
184     {
185         /* Lock must be taken to wait on conditional variable */
186         /*刚创建出线程，等待任务队列里有任务，否则阻塞等待任务队列里有任务后再唤醒接收任务*/
187         pthread_mutex_lock(&(pool->lock));
188
189
190         while ((pool->queue_size == 0) && (!pool->shutdown))
191         {
192             printf("thread 0x%x is waiting\n", (unsigned int)pthread_self());
193             //空闲线程等待任务，释放锁，返回时获得锁。
194             //有任务了，任务不为空，开始工作
195             pthread_cond_wait(&(pool->queue_not_empty), &(pool->lock));

```

```

196      /*清除指定数目的空闲线程，如果要结束的线程个数大于0，结束线程*/
197      if (pool->wait_exit_thr_num > 0)
198      {
199          pool->wait_exit_thr_num--;
200          /*如果线程池里线程个数大于最小值时可以结束当前线程*/
201          if (pool->live_thr_num > pool->min_thr_num)
202          {
203              printf("thread 0x%x is exiting\n", (unsigned int)pthread_self());
204              pool->live_thr_num--;
205              //解锁才退出
206              pthread_mutex_unlock(&(pool->lock));
207              pthread_exit(NULL);
208          }
209      }
210  }
211
212
213      /*如果指定了true，要关闭线程池里的每个线程，自行退出处理*/
214      if (pool->shutdown)
215      {
216          pthread_mutex_unlock(&(pool->lock));
217          printf("thread 0x%x is exiting\n", (unsigned int)pthread_self());
218          pthread_exit(NULL);
219      }
220      /*从任务队列里获得任务*/
221      task.function = pool->task_queue[pool->queue_front].function;
222      task.arg = pool->task_queue[pool->queue_front].arg;
223      pool->queue_front = (pool->queue_front + 1)%pool->queue_max_size;
224      pool->queue_size--;
225
226
227      /*通知可以有新的任务添加进来*/
228      //唤醒生产者，任务不满
229      pthread_cond_broadcast(&(pool->queue_not_full));
230
231
232      pthread_mutex_unlock(&(pool->lock));
233
234
235      /*执行任务*/
236      printf("thread 0x%x start working\n", (unsigned int)pthread_self());
237      pthread_mutex_lock(&(pool->thread_counter));
238      pool->busy_thr_num++; /*忙状态线程数加1*/
239      pthread_mutex_unlock(&(pool->thread_counter));

```

```

240     (*(task.function))(task.arg);                                /*执行回调函数任务
241     //task.function(task.arg);                                    /*执行回调函数任务*/
242
243
244     /*任务结束处理*/
245     printf("thread 0x%x end working\n", (unsigned int)pthread_self());
246     pthread_mutex_lock(&(pool->thread_counter));
247     pool->busy_thr_num--;                                          /*忙状态数减1*/
248     pthread_mutex_unlock(&(pool->thread_counter));
249 }
250
251
252 pthread_exit(NULL);
253 //return (NULL);
254 }
255
256
257 //主控线程方法
258 void *adjust_thread(void *threadpool)
259 {
260     int i;
261     threadpool_t *pool = (threadpool_t *)threadpool;
262     while (!pool->shutdown)
263     {
264         sleep(DEFAULT_TIME);                                      /*延时10秒*/
265         pthread_mutex_lock(&(pool->lock));
266         int queue_size = pool->queue_size;
267         int live_thr_num = pool->live_thr_num;
268         pthread_mutex_unlock(&(pool->lock));
269
270
271         pthread_mutex_lock(&(pool->thread_counter));
272         int busy_thr_num = pool->busy_thr_num;
273         pthread_mutex_unlock(&(pool->thread_counter));
274
275
276         /*任务数大于最小线程池个数并且存活的线程数少于最大线程个数时，创建新线程*/
277         if (queue_size >= MIN_WAIT_TASK_NUM && live_thr_num < pool->max_thr_num)
278         {
279             pthread_mutex_lock(&(pool->lock));
280             int add = 0;
281             /*一次增加DEFAULT_THREAD个线程*/
282             for (i = 0; i < pool->max_thr_num && add < DEFAULT_THREAD_VARY
283
284

```

```

283         && pool->live_thr_num < pool->max_thr_num; i++)
284     {
285         //是否有空位放新线程
286         if (pool->threads[i] == 0 || !is_thread_alive(pool->threads[i]))
287         {
288             pthread_create(&(pool->threads[i]), NULL, threadpool_thread, (void *)i);
289             add++;
290             pool->live_thr_num++;
291         }
292     }
293     pthread_mutex_unlock(&(pool->lock));
294 }
295
296
297 /*销毁多余的空闲线程*/
298 //忙线程2倍小于存活线程，控制最小有10个
299 if ((busy_thr_num * 2) < live_thr_num
300     && live_thr_num > pool->min_thr_num)
301 {
302     /*一次销毁DEFAULT_THREAD个线程*/
303     pthread_mutex_lock(&(pool->lock));
304     pool->wait_exit_thr_num = DEFAULT_THREAD_VARY;
305     pthread_mutex_unlock(&(pool->lock));
306
307
308     for (i = 0; i < DEFAULT_THREAD_VARY; i++)
309     {
310         /*通知处在空闲状态的线程,进行销毁*/
311         pthread_cond_signal(&(pool->queue_not_empty));
312     }
313 }
314 }
315 }
316
317
318 int threadpool_destroy(threadpool_t *pool)
319 {
320     int i;
321     if (pool == NULL)
322     {
323         return -1;
324     }
325
326

```

```

327     pool->shutdown = true;
328     /*先销毁管理线程*/
329     pthread_join(pool->adjust_tid, NULL);
330     for (i = 0; i < pool->live_thr_num; i++)
331     {
332         /*通知所有的空闲线程*/
333         pthread_cond_broadcast(&(pool->queue_not_empty));
334         pthread_join(pool->threads[i], NULL);
335     }
336     threadpool_free(pool);
337     return 0;
338 }
339
340
341 int threadpool_free(threadpool_t *pool)
342 {
343     if (pool == NULL)
344     {
345         return -1;
346     }
347     if (pool->task_queue)
348     {
349         free(pool->task_queue);
350     }
351     if (pool->threads)
352     {
353         free(pool->threads);
354         //获得锁，销毁锁
355         pthread_mutex_lock(&(pool->lock));
356         pthread_mutex_destroy(&(pool->lock));
357         pthread_mutex_lock(&(pool->thread_counter));
358         pthread_mutex_destroy(&(pool->thread_counter));
359         pthread_cond_destroy(&(pool->queue_not_empty));
360         pthread_cond_destroy(&(pool->queue_not_full));
361     }
362     free(pool);
363     pool = NULL;
364     return 0;
365 }
366
367
368 int threadpool_all_threadnum(threadpool_t *pool)
369 {
370     int all_threadnum = -1;

```

```

370     all_threadnum = pool->all_thr_num;
371     pthread_mutex_lock(&(pool->lock));
372     all_threadnum = pool->live_thr_num;
373     pthread_mutex_unlock(&(pool->lock));
374     return all_threadnum;
375 }
376
377
378 int threadpool_busy_threadnum(threadpool_t *pool)
379 {
380     int busy_threadnum = -1;
381     pthread_mutex_lock(&(pool->thread_counter));
382     busy_threadnum = pool->busy_thr_num;
383     pthread_mutex_unlock(&(pool->thread_counter));
384     return busy_threadnum;
385 }
386
387
388 int is_thread_alive(pthread_t tid)
389 {
390     int kill_rc = pthread_kill(tid, 0);
391     if (kill_rc == ESRCH)
392     {
393         return false;
394     }
395     return true;
396 }
397
398
399 /*测试使用，作成库时请注释掉下面代码*/
400 #if 1
401 void *process(void *arg)
402 {
403     printf("thread 0x%x working on task %d\n ",(unsigned int)pthread_self(),*(int *)arg);
404     sleep(5);
405     printf("task %d is end\n",*(int *)arg);
406     return NULL;
407 }
408 int main(void)
409 {
410     threadpool_t *thp = threadpool_create(3,100,12);    /*线程池里最小3个线程，最大100个线程，12秒超时*/
411     printf("pool initied");
412
413

```

```
414     int *num = (int *)malloc(sizeof(int)*20);
415     //int num[20];
416     int i;
417     for (i=0;i<10;i++)
418     {
419         num[i]=i;
420         printf("add task %d\n",i);
421         //添加任务
422         threadpool_add(thp,process,(void*)&num[i]);
423     }
424     sleep(10);
425     threadpool_destroy(thp);
426 }
427 #endif
```